Whether an action may take effect

A governance language for AI agents, and the conformance suite that tests it

Abstract. AI agents now cause external effects: they call tools, move data and delegate work to other agents. The rules that should govern those effects are written in prose, prompts, workflow code and access-control configuration. Standards such as XACML and ODRL give some of them a precise semantics, but both decide a request about a resource; the token here names no resource. It is an action with a kind and a risk, and the question is whether it may take effect. I present Loomground Governance, a small declarative language in which a governance policy is a graph of actors, gates and a single release point, the *master*. Evaluation assigns each gate one of five verdicts by priority selection, propagates verdicts along pipes as a join on a finite chain, and releases an action only when the effective verdict is `auto`. Execution is the host's, and the decision binds only where the host mediates every effect. Much of this is assembled from earlier work, and the paper says which parts.

The paper's contribution is how conformance to the language is tested. The semantics is published as an executable specification with 75 conformance vectors, whose runners check the reason for every rejection, compare the whole observation and test determinism under repeated and permuted runs. The suite is then tested itself: nine stubs that each break a runner rule and sixteen mutants that each break a rule of the semantics are scored through it. All nine stubs and fifteen mutants fail; the one that passes shows that no vector distinguished a `risk >=` guard from an equality. Two evaluators pass every vector and agree on eleven further probes, which is differential testing and not the independent interoperability the specification sets as its exit criterion. Each claim the language makes is set against the vectors that exercise it, and the three the suite does not reach are named.

I write the code and this text with Loomground agents, using Claude and Codex as tools, and I read and correct all of it.

1. Introduction

The governance question for an agent is not only what the system may do. It is whether this action may take effect, and under which observable conditions. An agent that drafts a document, decides a credit application and sends a payment performs three actions. They differ in consequence, in principal and in the obligation to involve a human.

Today the constraints on such actions live in formats that are not connected: legal and policy prose, system prompts, workflow logic, access-control configuration, model-specific guardrails, approval processes and audit logs. They share a vocabulary (“policy”, “human oversight”, “approval”, “prohibition”, “escalation”) but no shared, observable contract for when an agent’s action may take effect, against which two runtimes could be compared. I stated the problem bluntly in the project’s README: *“Every governance tool defines “policy”, “gate” and “verdict” its own way; nothing is comparable or checkable.”* [LG, README].

A policy in the language is short enough to show whole. This one is the conformance vector `draft-decide-run`:

```
actor  bot7
human  alice  role legal
gate   draft   risk low    grant bot7
gate   decide  risk high   grant bot7
reserve automated_decision by legal when risk >= high
cord bot7   -> draft
cord bot7   -> decide
cord draft  -> master
cord decide -> master
```

A routine low-risk token proposed by `bot7` at `draft` is released. A high-risk automated decision proposed at `decide` is reserved for the legal role, and the master withholds it, whatever grade `bot7` holds, provided an authorising grant exists at all. The language expresses the reservation and the referral; the human process that follows is the host's (§4.5). Section 3 defines the constructs and §4 the evaluation.

This paper treats the gap as a problem of language design and interoperability. It asks one question:

> **RQ.** Can governance constraints over agent actions be stated in a small declarative language whose semantics is precise enough that separately built evaluators can be tested for observational equivalence?

The paper makes five contributions.

1. **A language.** A graph model with four node classes, three cord types and a five-verdict chain that expresses authority, human reservation, prohibition, delegation, purpose and release.
2. **A semantics** that separates verdict assignment at one gate (priority selection) from propagation across gates (a join), with a single release rule and an ordered, verdict-labelled record.
3. **An executable specification**: normative prose, grammar, schemas, vocabularies and 75 conformance vectors, with a test that every feature reaches a vector and runners that check why an input is rejected, not only that it is.
4. **A suite shown to discriminate**, by stubs that break the runners' rules and by mutants that each break one rule of the semantics. The scores are in §6; the one mutant the suite does not catch names a gap in it.
5. **An evaluation** by replication and differential probing of two evaluators, which reproduces the suite and tests claims beyond it, with an account of what 75 of 75 establishes and of what it does not.

The paper makes no claim of legal compliance. The specification itself rules that out: *"Expressing a measure in this language does not, by itself, satisfy any legal obligation"* [LG, SPEC §1].

2. What is borrowed, and what is not

The split between deciding and enforcing is older than any policy language. ITU-T X.812 defines an access control decision function and an enforcement function that admits only what the decision function allows [X812, 3.4.3–3.4.4, 5.2.1]. The IETF named them policy decision point and policy enforcement point [RFC2753; RFC3198]; XACML's glossary keeps the mapping [XACML, glossary]. The master is the decision function for release. The enforcement point is the host's, under complete mediation [PROT].

Three choices here are XACML's. One verdict at a gate by priority selection is its `first-applicable`; propagation by the most restrictive is its `deny-overrides`; a public vector suite as the conformance criterion is its practice, with several hundred cases [XACML-CONF]. Two things differ. XACML's four decisions are unordered and the author picks one of twelve combining algorithms; here five verdicts form a chain and combine by one join, a restriction whose cost is every combination the chain cannot express. And XACML has no decision that refers an action to a human. A referral there is an obligation on a `Permit`, which the enforcement point *"SHALL permit access only if it understands and it can and will discharge those obligations"* [XACML, §7.2.1]; here the referral is the verdict. ODRL states permissions, prohibitions and duties over an asset and defines no evaluation semantics [ODRL; ODRL-FS]. Both decide a request about a resource; the token here names none.

Two further moves are twenty years old. Rei assigns priorities over policies [REI]; Ponder's `refrain` states what a subject must not do even where it holds the authorisation [PONDER]. Rei's engine, its authors record, "has not been designed to enforce the policies but only to reason about them" [TONTI, §4], and `refrain` is enforced at the subject. Here release is a value of the semantics, decided by one function outside every subject; whether it binds is the host's (§4.5). KAoS treats autonomy as a dial that policy bounds [KAOS; SCERRI]; here the dial is refused for reserved kinds, because a reservation does not read the grade.

The normative-systems tradition names what a reservation is. Grossi, Aldewereld and Dignum distinguish regimentation, where the design makes a violation impossible, from enforcement, where the institution reacts to one [GROSSI]; Boella and van der Torre separate norms that bind the actor from norms that bind the institution's own processing [BOELLA-PROC]. A reserved kind is a norm of the second sort, regimented, and within the model it plays the part of a Hohfeldian disability [JONES; SERGOT]: no delegation, grant or grade confers the power to release an action of that kind. Priority selection is the superiority relation of defeasible deontic logic [GOVERNATORI]. The join on a chain is the least general case of policy composition over the Belnap lattice [PBEL]; the design choice is the chain, and in particular that `human` sits below `refused`.

Three 2026 proposals occupy neighbouring ground. AgenticRei evaluates deontic policies at runtime, outside the model [AGENTICREI]. AgentBound composes three authorities into permitted, reviewed or denied before execution, with replayable receipts [AGENTBOUND]. OBPE authorises the typed operation, holds or denies, and proves, under stated conditions, that agent policy cannot widen a data owner's ceiling [OBPE]. Attenuation along delegation is proved or made unrepresentable elsewhere [OCAP; SECPAL]; here it is inherited and tested by vectors.

Searches of the ACM, IEEE and arXiv indexes and of the ISO/IEC, OASIS and W3C catalogues in October 2026 found none of four things: an outcome order in which reservation and prohibition differ from refusal; a referral that is itself a verdict; a typed policy graph as the policy object; a reservation that ignores the actor and the actor's grade. Three-valued orders are now common [AGENTBOUND; OBPE], and OBPE names an approval route on its hold as a parameter. A search that finds nothing proves no absence; these are the places where a counterexample would be most informative.

Three results bear on claims made here. A per-action policy cannot see a violation that lives in a sequence [PATHS]; `provenance` is a partial answer. User-authored permission policies blocked less overreach than per-action approval [YAN]; reservations here are authored on kind by the policy owner. Approval can be hijacked between understanding and execution [LOOP]; the `human` verdict binds to one token and is re-checked at the master (§4.4), and what a later human execution is bound to is the host's (§4.5).

ISO/IEC 22989 gives seven levels of automation and names only level 6 autonomy [ISO22989, 5.13]; a `grade` names latitude within external control. The oversight and logging controls of the standards corpus address the organisation, not a policy language [ISO42001; NIST-AI-100-1]. The exit criterion for interoperability [LG, SPEC §9] is the IETF's and the W3C's [RFC2026; W3C-PROCESS].

3. The language

I derived the language by decomposing "AI governance" into questions that can be answered at runtime, one construct per question (Table 1). I treat a construct that answers no row as a candidate for removal.

Table 1. Governance questions and the construct that answers each.

GOVERNANCE QUESTION	LOOMGROUND GOVERNANCE PRIMITIVE	SPEC
Who may act?	`actor` with an explicit authority cord (`grant`)	§3, §5.1
Where must governance occur?	`gate`	§3
What reaches the outside world?	the single `master` , reached only by `egress`	§3, §5.1, §7.3
What is being attempted?	token `kind` , observed by the host	§4
How severe is it?	token `risk` , computed by the gate, raise-only	§4
May an agent act unattended?	granted vs required `grade`	§6, §7.1
Must a human decide?	`reserve … by <role>` gives `reserved`	§6
Is the actor unauthorised?	no grant gives `refused`	§5.1, §7.1

GOVERNANCE QUESTION	LOOMGROUND GOVERNANCE PRIMITIVE	SPEC
Is the action forbidden whatever the authority?	`prohibit` gives `prohibited`	§6, §7.1
For whom does the agent act?	`on-behalf-of` gives the principal chain	§6
For what purpose?	`mandate`, `transfer … within`	§6
Can authority grow through delegation?	no-amplification and mandate-attenuation invariants	§6
Can the result be audited?	an ordered, verdict-labelled log	§7.4

A policy is a graph. Its four node classes are `actor`, `human`, `gate` and `master`; its three cord types are *authority* (actor to gate), *pipe* (gate to gate) and *egress* (gate to master) [LG, SPEC §3, §5.1]. An action enters as a *token* carrying `id`, `kind`, `risk`, `party`, `provenance`, `reversibility`, `uncertainty` and `tags` [LG, `language-card.json`]. A guard ranges over six token properties — `kind`, `risk`, `reversibility`, `uncertainty`, `party`, `tags` — and computes nothing [LG, SPEC §6]. Of these, `risk` is not declared by the supplier. The gate computes it from its governed table, and a gate's floor may raise it above the declared hint [LG, SPEC §4]. A risk guard therefore reads the effective risk at the gate; `reversibility` and `uncertainty` read declared values. A guard never ranges over `grade`, which is a configuration attribute rather than a token field [LG, `language-card.json`, litmus]. The conformance vector `draft-decide-run` of §1 is written in this syntax. The delegation keyword is `on-behalf-of`; the observation schema projects it as the field `on_behalf_of`.

4. Semantics

An action becomes a token that enters the graph at a source gate. The graph is evaluated once per activation, and only the master can release [LG, SPEC §5.2, §7].

4.1 Well-formedness

An ill-formed graph, or a token lacking a required field or bearing a risk outside its domain, denotes $\perp$. $\perp$ is not a verdict; it is the absence of an evaluation, distinct from `auto`, the bottom of the chain in §4.3. A conforming implementation *"MUST reject a token lacking a required field or bearing a risk outside its domain"*; such a token *"activates no node and produces no log entry"*, and the master withholds [LG, SPEC §4, §7.3]. The default risk domain, fixed by the token schema, is `low < medium < high < critical` [LG, schema/token.schema.json].

4.2 Verdict assignment at one gate

Each activated gate assigns exactly one verdict by priority selection [LG, SPEC §7.1]:

1. `prohibited` if the token matches a prohibition;

2. `refused` if the proposing actor holds no grant at the gate that authorises the token (a grant may be narrowed to kinds and risk levels);
3. `reserved` if the token matches a reservation, its risk guard compared against the gate-computed risk, floor included (§3);
4. otherwise `auto` or `human` by the grade rule: `auto` if the actor's granted grade G is at least the gate's required grade R, or the gate declares no R; `human` if G is lower, or the actor is ungraded at a graded gate. A required grade may be declared only at a source gate, and the grade compared is the proposing actor's.

Prohibition therefore comes before the authority check. Prohibitions and reservations match the token's kind and its declared guards, never the actor or the actor's grade. No grant or grade reaches a reservation. A reserved action is withheld from every actor and left to the role it names; what that role does is the host's process (§4.5). Actor-independence concerns the match, not the verdict: an actor without an authorising grant is `refused` before any reservation is considered.

4.3 Propagation

Along pipes, a gate's effective verdict is the join of its own verdict and the effective verdicts of every gate that pipes into it and is activated in the same activation. The verdicts form a finite chain, and the join is the maximum:

$$\text{auto} \sqsubseteq \text{human} \sqsubseteq \text{refused} \sqsubseteq \text{reserved} \sqsubseteq \text{prohibited}$$

$$\text{eff}(g) \;=\; \text{own}(g) \sqcup \bigsqcup \{\, \text{eff}(h) \colon h \xrightarrow{\text{pipe}} g,\ h \text{ activated} \,\}$$

A gate that is not activated contributes no term, and a gate with no activated predecessor keeps its own verdict, since the empty join is `auto`, the bottom of the chain [LG, SPEC §7.2]. Pipes are acyclic, and the maximum on a chain is associative, commutative and idempotent, so `eff` is well-defined and the effective verdicts do not depend on traversal order. The log is a different matter: it records evaluation order, and the pipes may admit more than one topological order. The specification fixes one. Among the gates whose activated predecessors are already logged, the gate declared earliest is evaluated next, and *"a conforming implementation's log MUST match the one this rule produces"* [LG, SPEC §7.4]. The priority order at a gate and the join order disagree on the `refused` / `reserved` pair; the specification allows this, since the two orderings *"need not agree"* [LG, SPEC §7.1]. Both verdicts withhold. The priority order decides which verdict a gate assigns; the join order decides which reason a withholding should record (the most restrictive contributing verdict [LG, SPEC §7.3]), not how strongly it withholds. `reserved` above `refused` is a diagnostic precedence, not a stronger restriction: a withholding to which a reservation contributes is recorded as reserved, which is the reason a role must act, rather than as a missing grant.

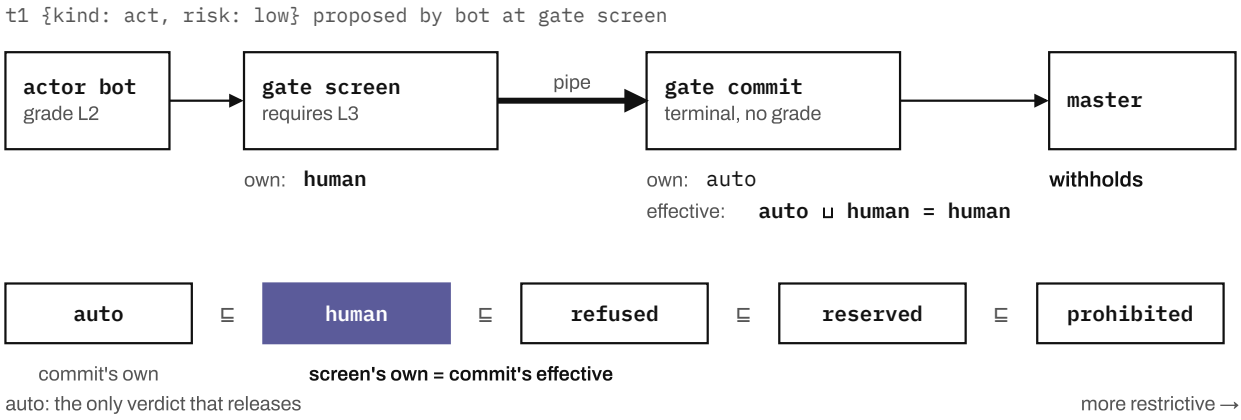


Figure 1. The verdict order, and propagation read off it, in the conformance vector grade-join. Actor bot, granted grade L2, proposes a token at source gate screen, which requires L3, so screen's own verdict is human. Gate commit declares no grade and no reservation, so its own verdict is auto; its effective verdict is the join of the two, human, and the master withholds.

4.4 Release and record

The master releases a terminal gate's action if and only if its effective verdict is `auto` and every egress obligation is attached; every other verdict, and ⊥, withholds [LG, SPEC §7.3]. A declared obligation is attached by its declaration; its discharge is outside the language. The master decides each terminal gate independently, so an activation that reaches two terminal gates may be released at one and withheld at the other [LG, SPEC §7.3]. Every activated gate produces one log entry, in evaluation order. The entry carries the effective verdict, the declared token and the host-observed facts behind its `kind` and `risk`, so that a later re-examination sees the claim and the observation [LG, SPEC §7.4]. Evaluation is a deterministic function of the token, the host-observed facts from which its `kind` and `risk` are derived, the risk table and the policy, declaration order included. Repeated runs yield identical observations; a reordering of the declarations leaves every verdict and every master decision unchanged, and changes the log only where the tie-break of §4.3 then chooses differently [LG, SPEC §7.5].

4.5 What the language leaves to the host

The specification excludes execution, scheduling, storage and communication from the language [LG, SPEC §10]. It still decides release, because release is a verdict, not an act. Two places sit close to that boundary. Risk is *computed* by the gate from a governed, versioned, signed table whose content is policy [LG, SPEC §4], so some computation is inside the language. A temporal reservation declares a duration and a resolution on elapse; measuring the interval and acting on the resolution are the host's [LG, SPEC §6; `conformance/README.md`]. Neither changes the release rule: within the specification only `auto` releases, and any later activation is evaluated afresh, with no carry-over from the one that withheld [LG, SPEC §7.3]. The authenticity of `party` and the integrity of `tags` are likewise the host's [LG, SPEC §4, §10].

A withheld action proceeds as follows. In the example of §1, the high-risk `automated_decision` at `decide` is reserved for the legal role; the master withholds, and the log records the token, the

observed facts and `reserved`. Nothing in the language then approves it, because a reservation reads the kind and not the actor: the same token proposed by `alice` would be reserved as well, given a grant that authorises her at that gate, and `refused` without one. The action is the role's to take, outside the language, and the record is what the role takes it on. Where the withholding is `human` — a grade too low at a graded gate — an actor holding the required grade may propose the token again, and it is evaluated from the start against its own facts. In both cases the specification draws no edge from the withholding to a later activation: *"A reader MUST NOT assume the reserved action's constraints bind the follow-on activation"*; any such binding is policy [LG, SPEC §7.3]. A `redress` declaration can name a later re-examination as owed without adding that edge. The language has no transition that turns a withheld decision into a release within the same activation; it withholds, records, and evaluates the next proposal afresh. An approval that carried state across activations would be a workflow, and workflows are the host's.

5. Method

5.1 One semantics in several representations

The semantics is stated in several representations (Table 2). The project's rule is that a feature counts as complete only when *"the spec, the grammar, the schemas, the vocabulary, `llms.txt`, and a conformance vector agree"* [LG, CONTRIBUTING.md]. A test enforces part of that rule in continuous integration: every declaration, every projected schema field and every grammar keyword must be exercised by at least one conformance vector [LG, `tests/lockstep_meta_test.py`]. It checks coverage, not that the representations say the same thing. Where they must agree, they do. The guard domain has the same six fields in the prose, the grammar, the vocabulary and the five vectors that use `reversibility` or `uncertainty`; the prose, the syntax and the language card carry the manifest's version.

Table 2. Representations of the semantics, at specification version 0.11.2 (commit `081f1cf`).

REPRESENTATION	ARTEFACT	COUNT
Normative prose	`spec/SPEC.md`, which governs on conflict	12 sections
Concrete syntax	`spec/SYNTAX.md`	—
Grammar	`grammar/loomground.ebnf`, `grammar/tree-sitter/`	2
Data schemas	token, patch, observation, transport	4
Vocabularies	verdicts, grades, risk, cords, guard domain, rejection reasons and others	12
Agent surface	`language-card.json`, `llms.txt`, `AGENTS.md`	3

REPRESENTATION	ARTEFACT	COUNT
Test oracle	`conformance/manifest.json` and vectors	75

5.2 The conformance suite

The manifest lists 75 vectors of four kinds (Table 3).

Table 3. The conformance suite at specification version 0.11.2.

KIND	COUNT	WHAT THE VECTOR FIXES	WHAT THE RUNNER COMPARES
patch	40, of which 27 carry a run	the input and the projection a conforming implementation must produce; with a run, the verdict at each gate, the master decision and the ordered log	the whole observation, so an extra, missing or misordered member fails it
negative	31: 27 at apply, 4 at parse	the stage at which the input is rejected, and the reason, one of twenty codes in `vocabulary/reject-reasons.json`	the stage and the reason
token	2	whether a token is well-formed or defective	the classification
determinism	2	a repeated run, and runs against reorderings of the declarations, each with the log its order fixes	verdicts and decisions unchanged; each log as its order fixes

Patch vectors cover release, reservation, multi-hop propagation, fan-out and fan-in, authority at piped gates, invalid tokens inside a run, information-flow guards, prohibition, grades, quorum, temporal reservation, delegation chains, mandates and egress obligations. They also compose situations no single declaration exercises: a fan-out and fan-in that admits two evaluation orders, declared once in each; a prohibition with a high grade and a reservation on the same kind; a token whose declared risk differs from the computed risk, so the log entry carries both.

The conformance README binds every runner to four rules [LG, `conformance/README.md`]. A runner checks the reason for a rejection as well as its stage. It treats an exception raised while parsing as a correct rejection only for a vector declared to fail at parse. It compares the whole observation, so an extra, missing or misordered member anywhere fails the vector. And it executes the determinism vectors: repeats must agree with the expected verdicts, decisions and log, and each reordering must agree with the same verdicts and decisions and with its own log. A run may mark a token as deliberately invalid; it then contributes no expected entry and no log entry, and *“a runner MUST fail the vector if a token so marked in fact validates”* [LG, `conformance/README.md`]. A projected quorum

or duration is evidence that the declaration is read correctly, not that distinct approvals are authenticated or that a timer fires.

5.3 Two evaluators

`loomground-ref` is a stdlib-only evaluator labelled as implementing specification 0.11.2 [REF, README.md]. `loomground-solver` evaluates the language as one component of a larger reasoning service [SOLVER]. I wrote both, in the same project, with the same agents. `loomground-ref` says so in its README and states the specification's independence criterion as open; neither repository claims independence from the other [REF, README.md; SOLVER, `docs/contracts/loomground-compatibility.md`]. The two evaluators share one structure: reachability over pipes, a memoised join over activated predecessors, and a topological order with ties broken by declaration order, the rule SPEC §7.4 makes normative. Their agreement is therefore evidence of *differential testing*, not of independent interoperability.

5.4 Differential testing

The same input is given to both evaluators, and their observations are compared with each other and with what the specification requires (Figure 2). Agreement reproduces a case; disagreement is a finding, resolved in the specification, the vector or an evaluator. Beyond the 75 vectors, I wrote probes aimed at priority and precedence, and at delegation bounds. For each probe, the comparison script checks the two evaluators against each other on the stage at which the policy is accepted or rejected and, for a run, on each gate's verdict, the master decision and the sequence of logged verdicts. Agreement with the specification is my reading of each case against the cited sections.

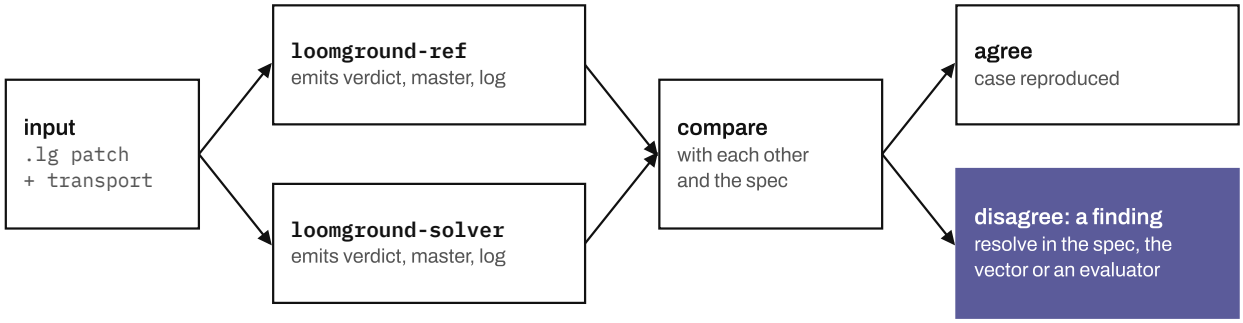


Figure 2. Differential testing. The same input goes to both evaluators, and their observations (verdict per gate, master decision, ordered log) are compared with each other and with the specification. A disagreement is a finding, resolved in the specification, the vector or an evaluator.

All runs use loomground 0.11.2 (tag `v0.11.2`, `081f1cf`), loomground-governance 0.12.0 (`648cb89`), loomground-ref `2417fee`, loomground-solver `fc7f0f4` and loomground-deontic 0.2.1 (`319e218`) [DEONTIC], run on 2026-10-10. The mutant battery of §6.3 ran on 2026-10-20 at loomground-ref `0bd1e7e` against the same standard. The standard in loomground-governance is pinned to loomground v0.11.2, and its copy of the vectors is byte-identical to loomground's [GOV, CHANGELOG.md, `standard/CANONICAL`]. The solver declares loomground-governance 0.12.0 as a

dependency, so its own conformance gate runs the released kit and meets the full suite of 75 vectors [SOLVER, `pyproject.toml`]. Commands and raw output ship with the paper in its kit [KIT].

6. Results

6.1 Replication

Both evaluators reproduce the full suite. `loomground-ref` passes 75 of 75 vectors against both copies of the standard, and 49 of 49 machine-readable checks, including a check that every non-rejecting input is accepted by its own parser. `loomground-solver` passes 75 of 75 through the conformance kit shipped with loomground-governance [GOV], and 75 of 75 through `loomground-ref`'s runner, which it meets through an adapter that translates calls and holds no language logic. The kit's bundled engine also passes 75 of 75.

The runners reject evaluators that pass by accident. Through the kit, a stub that raises on every input scores 0 of 75, and so does one that rejects every patch at apply. `loomground-ref` ships nine stubs, each breaking one rule of its runner [REF, `tests/test_stub_rejection.py`]. Two reject every input, one by raising and one at the apply check. One gives a wrong reason at the right stage. Two add or drop an observation key. One flips its log order between calls. One reports an undeclared gate. Two move a parse-stage rejection to the wrong place. They score between 5 and 71 of 75.

6.2 Agreement beyond the suite

Eleven probes test precedence and delegation beyond the vectors. Both evaluators agree on all eleven, and in each case the outcome is the one the specification requires (Table 4). One of them, H3a, also has a counterpart in the suite, `prohibit-grade-reservation-precedence`.

Table 4. Probes on which both evaluators agree with the specification.

PROBE	SPECIFICATION REQUIRES	BOTH EVALUATORS
H1, H1′, H1″	a high-risk automated decision under a reservation is `reserved`, including a low-risk token raised by the gate's floor and an L6 actor at an L2 gate	`reserved`, withhold
H3a	a prohibition beats a grant, a high grade and a reservation	`prohibited`, withhold
H3b, H3b′	an ungranted actor is `refused`; two activations at one gate are both logged in order	`refused`; log `reserved`, `refused`
H2a–H2c	delegation that amplifies grade or risk, or forms an `on-behalf-of` cycle, is rejected at apply	rejected at apply

PROBE	SPECIFICATION REQUIRES	BOTH EVALUATORS
H2d	delegation that attenuates grade is accepted	accepted
H3c	a cord from an actor directly to the master is rejected at apply	rejected at apply

6.3 Mutants

The stubs of §6.1 break rules the runners check; none of them gets a verdict wrong. A second battery does. Each of sixteen mutants changes one rule of the passing evaluator: a step of the priority order, the join, the release rule, a guard, the narrowing of a grant, the risk floor, the tie-break, the handling of an invalid token, or one of four delegation bounds. Each is scored through the same runner (Table 5). Fifteen score below the total, and the vectors that fail each are named, so the suite's sensitivity to each alteration is a count rather than a claim. The counts are small. Five mutants are caught by a single vector and five more by two; where the suite reaches a rule it does so with little redundancy, and for five mutants the loss of one vector would leave them uncaught. A caught mutant shows sensitivity to that alteration, not coverage of every violation of the rule it breaks. One mutant survives. No vector runs a token above the threshold of a `risk >=` guard, so an evaluator that reads `>=` as `=` passes all 75. The test records the gap with the vector that closes it, `guard-risk-above-threshold`, released in loomground 0.11.4 after this evaluation. The record holds only while that vector is absent; against 0.11.4 the mutant is caught by that one vector [REF, `tests/test_semantic_mutants.py`].

Table 5. Sixteen mutants of the passing evaluator, each breaking one rule of the semantics, scored through the strict runner at specification version 0.11.2.

MUTANT	RULE IT BREAKS	SCORE	CAUGHT BY
`reserved-before-refused`	priority step 2 before 3	74 of 75	`refused-precedence`
`refused-before-prohibited`	priority step 1 before 2	74 of 75	`refused-precedence`
`grade-clears-reservation`	reservation independent of the actor	73 of 75	`grade-reserved-precedence`, `reserve-grant-independence`
`ungraded-actor-auto`	ungraded actor fails closed	74 of 75	`grade-ungraded-at-graded`
`no-authority-downstream`	authority checked at every gate	71 of 75	`pipe-authority-downstream`, `topo-fanout-fanin-declared-order`, `topo-fanout-fanin-declared-order-variant` and 1 more

MUTANT	RULE IT BREAKS	SCORE	CAUGHT BY
`risk-floor-ignored`	gate floor raises the effective risk	74 of 75	`record-declared-and-observed`
`guard-ge-as-eq`	`>=` guard on an ordered property	75 of 75	none
`grant-risk-unnarrowed`	grant narrowed to risk levels	73 of 75	`grant-narrowing`, `reject-delegation-risk-amplify`
`join-least-restrictive`	join is the maximum on the chain	70 of 75	`grade-join`, `pipe-authority-downstream`, `topo-fanout-fanin-declared-order` and 2 more
`release-on-human`	only `auto` releases	72 of 75	`grade-human`, `grade-ungraded-at-graded`, `grade-join`
`log-by-name`	tie-break by declaration order	68 of 75	`draft-decide-run`, `grade-join`, `guard-reversibility` and 4 more
`invalid-token-evaluated`	invalid token denotes ⊥	71 of 75	`token-validation`, `risk-enum-values`, `transport-invalid-token` and 1 more
`grade-amplification-allowed`	no-amplification over grade	73 of 75	`reject-delegation-grade-amplify`, `reject-delegation-grade-from-nothing`
`risk-amplification-allowed`	no-amplification over granted risk	73 of 75	`reject-delegation-risk-amplify`, `reject-delegation-ungranted-delegator`
`mandate-widening-allowed`	mandate attenuation along delegation	73 of 75	`reject-mandate-widen`, `reject-mandate-from-nothing`
`obo-cycle-allowed`	acyclic on-behalf-of	74 of 75	`reject-obo-cycle`

7. Discussion

7.1 What 75 of 75 establishes

An evaluator that passes the suite agrees with the specification on the inputs the vectors contain, under the rules its runners apply; it is not thereby shown correct on inputs outside them. Table 6 sets each claim the language makes against the vectors that exercise it.

Table 6. Each claim the language makes, the evidence the suite provides, and what it does not reach.

CLAIM	EVIDENCE IN THE SUITE	STATUS
Reservations are actor-independent	`reserve-grant-independence`, `grade-reserved-precedence`	exercised
Missing authority fails closed	`refused-precedence`, `grant-narrowing` at source gates; `pipe-authority-downstream` at a piped gate	exercised
Invalid inputs fail closed	`token-validation`; `transport-invalid-token` and `transport-invalid-token-only` inside a run	exercised
Delegation is bounded	10 negative and 6 positive vectors with a delegation binding, and 3 negative transfer vectors; runners check the stage and the reason of each rejection	exercised
Restrictions compose	single-hop pipes; `pipe-fanout`, with fan-out, fan-in and a two-hop path; `topo-fanout-fanin-declared-order`, where a fan-in gate joins `refused` from one of two incomparable predecessors	exercised
Prohibitions override authority	`prohibit-tags`, `refused-precedence`; `prohibit-grade-reservation-precedence`, which combines a prohibition with a maximal grade and a same-kind reservation	exercised
Evaluation is auditable	log verdicts and order in every run; `record-declared-and-observed`, whose log entry carries the declared token and the observed facts	exercised
The record is tamper-evident	none: the specification requires tamper-evidence and leaves the mechanism to the host [LG, SPEC §7.4]	outside the specification

CLAIM	EVIDENCE IN THE SUITE	STATUS
A reservation withholds release without a workflow engine	`reserved` withholds at the master	exercised
Quorum and temporal reservations are enforced	quorum and duration project as declared, which shows the declaration is read; authenticating parties and measuring elapse are the host's [LG, SPEC §10]	not tested
Verdicts and release decisions are order-independent	`topo-fanout-fanin-declared-order` and its variant, which declare two incomparable gates in opposite orders; `determinism-topo-tie`, a three-way tie under three reorderings; verdicts and decisions agree, and each log follows SPEC §7.4	exercised
Decisions are deterministic	`determinism-repeat-permute`, `determinism-topo-tie`: repeated runs, and runs against reordered declarations	exercised
A `>=` guard on risk is read as an order	`guard-risk-ordering` projects the guard; no run places a token above its threshold, and the mutant that reads `>=` as `=` passes every vector (§6.3)	not tested

Ten of the thirteen claims are exercised, because the suite composes the situations they depend on. Its vectors put invalid tokens inside a run, leave the actor ungranted at a piped gate, give gates two successors and two predecessors, admit two evaluation orders and fix the log for each, and combine a prohibition with a high grade and a same-kind reservation.

Three are not, and the table says which. A quorum or a duration projects as declared: that shows the declaration is read, not that distinct approvals were authenticated or that a timer fired, and no vector reaches either. The specification requires tamper-evidence but leaves its mechanism to the host, so there is nothing for a vector to check. Those two sit with the host, and a reader who needs them needs the host's evidence rather than this suite's. The third is the suite's own, found by the mutants of §6.3: the `>=` guard on risk is run only at its threshold at 0.11.2; the vector that closes the gap is in loomground 0.11.4.

The runners make the count mean what it says (§5.2, §6.1), and the mutants of §6.3 say which alterations the vectors catch and which one they do not.

7.2 Specified properties and consistency checks

At the points checked, the normative text agrees with itself and with its surroundings (§5.1). The continuous-integration test holds coverage, not agreement, so what is stated here is a reading of

those points and not a proof over the whole text. The conformance README states that the repository carries the language and no implementation of its own, and that any implementation shows conformance in the same way, by reproducing every vector. Every rejection a negative vector pins has a code in a closed vocabulary of twenty, each documented against the section of the specification that makes the case ill-formed, so the reason for a rejection is machine-checkable [LG, `vocabulary/reject-reasons.json`].

The record and its reproducibility are fixed as well. Effective verdicts do not depend on traversal order, because the join is associative, commutative and idempotent. The log does, and SPEC §7.4 settles it with a total rule: among the gates eligible next, the one declared earliest. Two conforming implementations given the same token, the same host observations and risk table, and the same policy therefore produce the same log, gate for gate and verdict for verdict. SPEC §7.5 states three rules for determinism. A repeated run yields an identical observation. A reordering of the declarations yields the same verdicts and the same master decision. The log under a reordering is unchanged where the activated gates admit a single topological order; otherwise it is the log the tie-break fixes for the new order. The determinism vectors test all three.

7.3 Independence

The specification's exit criterion, aligned with the IETF and W3C processes [RFC2026; RFC6410; W3C-PROCESS], is two implementations, neither derived from the other, each passing every vector. The conformance README sharpens it: the criterion is met when *"an implementation produced without access to this project's implementations reproduces every vector"*. This is the one criterion the present evaluation cannot meet by construction: the two evaluators were written in one project, and their agreement is differential testing (§5.3). The criterion names its own test. A third party builds an implementation from the specification, syntax, schemas and vocabulary alone, with the vectors withheld until the implementation is frozen, and then runs it once against all 75.

7.4 Scope and limitations

Conformance is a property of an implementation, not a finding of legal compliance. Separation of duty holds only relative to an external binding of `party` to an authenticated identity, and tag guards only relative to the integrity of tag assignment. The language assumes `kind` is observed by the host and `risk` computed by the gate from a governed table; a host that lets actors declare `kind` breaks that premise. The master's `act` or `withhold` is a decision; enforcement requires a host adapter with exclusive control of the effect. Finally, the probes are mine, and a different set would probe different combinations.

8. Conclusion

Governance constraints over agent actions can be stated in a small declarative language with a precise, testable semantics: priority selection at a gate, a join across gates, a single release rule, and a record whose order the declarations fix. An executable specification with 75 vectors exercises

most of the claims the language makes, including the order-independence of verdicts and the determinism of evaluation under repeated and reordered runs, and names the three it does not reach. All nine stubs that break the runners' rules and fifteen of the sixteen mutants that break the semantics score below the total, so the suite is known to fail a defective evaluator and not only to pass a conforming one. The one mutant that survived is the finding about the suite itself. Every run against a `risk >=` guard sat at the threshold, so the suite did not distinguish an ordered comparison from an equality until mutation testing exposed an evaluator that treats `>=` as one. The 0.11.2 suite the paper evaluates does not distinguish the two readings; the vector that does is in loomground 0.11.4. Both evaluators pass every vector and agree on eleven further probes. The lesson is methodological. A conformance suite establishes what its vectors compose and what its runners check; both belong to the specification, not to the implementations it tests. A suite never tried against a defective evaluator has not been shown to do either. What remains is the specification's own exit criterion: an implementation built without access to the existing two that reproduces every vector.

Appendix. Reproduction

The commands need a POSIX shell, Python 3.10 or later and git. `KIT` is the directory that holds this paper and its scripts. The kit is published with the paper [KIT]; its script `reproduce.sh historical WORKDIR` clones the five repositories at the pins, builds a virtual environment and writes every output under `audit/historical/`, and the listing below is what it runs.

```
KIT=/path/to/kit
for p in loomground:081f1cf loomground-governance:648cb89 \
         loomground-ref:2417fee loomground-solver:fc7f0f4 \
         loomground-deontic:319e218; do
  r=${p%%:*}; c=${p#*:}
  git clone -q https://github.com/flxk1/$r && git -C $r checkout -q $c
  echo "$r $(git -C $r rev-parse HEAD)"
done
git -C loomground-ref worktree add -q ../loomground-ref-mutants 0bd1e7e   # the mutant
battery (§6.3)
python3 -m venv .venv && . .venv/bin/activate
pip install ./loomground-governance ./loomground-deontic jsonschema

# Vector counts (Table 3, §5.2)
python3 $KIT/audit/published/count_vectors.py loomground HEAD

# Replication
(cd loomground-ref    && python3 tools/verify.py ../loomground)
(cd loomground-ref    && python3 tools/verify.py ../loomground-governance/standard)
(cd loomground-ref    && LOOMGROUND_ROOT=../loomground python3
tests/test_machine_readable.py)
(cd loomground-solver && PYTHONPATH=src python3 tools/run_loomground_conformance.py)
(cd loomground-ref    && LOOMGROUND_REF=. PYTHONPATH=../loomground-solver/src \
   python3 $KIT/audit/published/solver-ref-adapter/run_ref_verify.py ../loomground)
python3 $KIT/audit/published/kit-bundled-engine.py loomground-governance

# Probes and runner behaviour
(cd loomground-ref    && LOOMGROUND_ROOT=../loomground python3
tests/test_stub_rejection.py)
(cd loomground-ref-mutants && LOOMGROUND_ROOT=../loomground python3
tests/test_semantic_mutants.py)   # Table 5
export PYTHONPATH=loomground-ref:loomground-solver/src
python3 $KIT/run_experiments.py --series H   # Table 4
python3 $KIT/audit/kit-mock-evaluators.py    # stub scores through the kit

# Quoted spans (re-derive quote-grounding.json at the pinned commits)
python3 $KIT/ground_quotes.py loomground loomground-ref $KIT/paper-I.md
```

The outputs of these commands, run on 2026-10-10, ship in the kit under `audit/published/`: `vector-counts.out`, `ref-verify.out`, `ref-verify-governance.out`, `ref-machine-readable.out`, `solver-conformance.out`, `solver-ref-verify.out`, `kit-bundled-engine.out`, `ref-stub-rejection.out`, `probes.out` and `kit-mock-evaluators.out`; `ref-semantic-mutants.out`, run on 2026-10-20 at loomground-ref `0bd1e7e`, is the output of Table 5. The probe battery behind Table 4 produces the following (condensed):

```
probe  case                              ref / solver
H1     DPO reservation, risk high         reserved, withhold (both)
H1'    risk low, gate floor high          reserved, withhold (both)
H1''   L6 >= required L2                  reserved, withhold (both)
H3a    prohibit beats L6 + reservation    prohibited, withhold (both)
H3b    ungranted actor                    refused, withhold (both)
H3b'   two activations, one gate          log [reserved, refused] (both)
H2a    grade amplification L3 under L2    reject@apply (both)
H2b    risk amplification                 reject@apply (both)
H2c    on-behalf-of cycle                 reject@apply (both)
H2d    attenuation L2 under L3            accepted (both)
H3c    agent -> master                    reject@apply (both)
```

References

- [LG] Loomground specification repository, https://github.com/flxk1/loomground, specification version 0.11.2, tag `v0.11.2`, commit `081f1cf`: `spec/SPEC.md`, `spec/SYNTAX.md`, `conformance/`, `schema/`, `vocabulary/`, `vocabulary/reject-reasons.json`, `language-card.json`, `tests/lockstep_meta_test.py`, `README.md`, `CONTRIBUTING.md`.
- [GOV] loomground-governance, https://github.com/flxk1/loomground-governance, release 0.12.0, commit `648cb89`: `standard/`, `standard/CANONICAL`, the conformance kit, the bundled engine `skills/loomground/loomground.py`, `CHANGELOG.md`.
- [REF] loomground-ref, https://github.com/flxk1/loomground-ref, commit `2417fee`: `loomground.py`, `tools/verify.py`, `tests/test_machine_readable.py`, `tests/test_stub_rejection.py`, `README.md`; `tests/test_semantic_mutants.py` at commit `0bd1e7e`, `docs/provenance.md`.
- [SOLVER] loomground-solver, https://github.com/flxk1/loomground-solver, commit `fc7f0f4`: `src/loomground_solver/loomground.py`, `tools/run_loomground_conformance.py`, `pyproject.toml`, `docs/contracts/loomground-compatibility.md`.
- [DEONTIC] loomground-deontic, https://github.com/flxk1/loomground-deontic, release 0.2.1, commit `319e218`, the runtime dependency the solver's `pyproject.toml` declares.
- [AGENTBOUND] A. Kaul, Q. Lan, P. Gupta, "Behavioral Governance for Autonomous AI Agents: The AgentBound Framework", arXiv:2606.30970v2, submitted 29 June 2026, revised 1 July 2026.
- [AGENTICREI] A. Joshi, T. Finin, K. P. Joshi, L. Kagal, "Deontic Policies for Runtime Governance of Agentic AI Systems", arXiv:2606.19464, 17 June 2026; to appear, IEEE ICWS 2026.
- [BOELLA-PROC] G. Boella, L. van der Torre, "Substantive and procedural norms in normative multiagent systems", *Journal of Applied Logic* 6(2), 152–171, 2008, doi:10.1016/j.jal.2007.06.006.
- [GOVERNATORI] G. Governatori, A. Rotolo, "BIO logical agents: Norms, beliefs, intentions in defeasible logic", *AAMAS Journal* 17(1), 36–69, 2008, doi:10.1007/s10458-008-9030-4.

- [GROSSI] D. Grossi, H. Aldewereld, F. Dignum, “Ubi Lex, Ibi Poena: Designing Norm Enforcement in E-Institutions”, *COIN II*, LNCS, 101–114, 2007, doi:10.1007/978-3-540-74459-7_7.
- [ISO22989] ISO/IEC 22989:2022, *Information technology — Artificial intelligence — Artificial intelligence concepts and terminology*, 1st ed., 2022-07, ISO/IEC JTC 1/SC 42. Defs 3.1.5, 3.1.16, 3.5.6; clause 5.13 and Table 1.
- [ISO42001] ISO/IEC 42001:2023, *Information technology — Artificial intelligence — Management system*, 1st ed., 2023-12. A.6.2.8 event logs; A.9.2 responsible use.
- [JONES] A. J. I. Jones, M. Sergot, “A Formal Characterisation of Institutionalised Power”, *Logic Journal of the IGPL* 4(3), 427–443, 1996, doi:10.1093/jigpal/4.3.427.
- [KIT] F. Krone, *Whether an action may take effect* — paper and reproduction kit, https://github.com/flxk1/thoughts, directory `whether-an-action-may-take-effect`, tag `whether-an-action-may-take-effect/v1.0.1`.
- [KAOS] A. Uszok, J. M. Bradshaw, R. Jeffers et al., “KAoS policy and domain services: toward a description-logic approach to policy representation, deconfliction, and enforcement”, *POLICY 2003*, 93–96, doi:10.1109/POLICY.2003.1206963.
- [LOOP] Kumar, “Loopjacking: Hijacking Human-in-the-Loop Approval”, arXiv:2609.21081, Sept. 2026.
- [NIST-AI-100-1] NIST AI 100-1, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, January 2023, doi:10.6028/NIST.AI.100-1. MANAGE 2.4, MAP 3.5, GOVERN 3.2.
- [OBPE] M. Millstone, T. Akidau, J. Brüderl, M. Pekker, “If Agents Were Angels, No Governance Would Be Necessary: Out-of-Band Policy Enforcement at a Trusted Tool Boundary”, arXiv:2608.27646, 27 Aug 2026.
- [OCAP] M. S. Miller, *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*, PhD thesis, Johns Hopkins University, 2006.
- [ODRL] W3C, *ODRL Information Model 2.2*, W3C Recommendation, 15 February 2018.
- [ODRL-FS] N. Fornara, V. Rodríguez-Doncel, B. Esteves, S. Steyskal, B. W. Smith, *ODRL Formal Semantics*, W3C ODRL Community Group Draft Report, 2025.
- [PATHS] Kaptein, Khan, Podstavnychy, “Runtime Governance for AI Agents: Policies on Paths”, arXiv:2603.16586, March 2026.
- [PBEL] G. Bruns, M. Huth, “Access control via Belnap logic: Intuitive, expressive, and analyzable policy composition”, *ACM TISSEC* 14(1), Article 6, 2011, doi:10.1145/1952982.1952991.
- [PONDER] N. Damianou, N. Dulay, E. Lupu, M. Sloman, “The Ponder Policy Specification Language”, *POLICY 2001*, LNCS 1995, 18–38, doi:10.1007/3-540-44569-2_2.
- [PROT] J. H. Saltzer, M. D. Schroeder, “The Protection of Information in Computer Systems”, *Proceedings of the IEEE* 63(9), 1975.
- [REI] L. Kagal, T. Finin, A. Joshi, “A Policy Language for a Pervasive Computing Environment”, *POLICY 2003*, 63–74, doi:10.1109/POLICY.2003.1206958.
- [RFC2026] S. Bradner, *The Internet Standards Process — Revision 3*, RFC 2026, IETF, 1996.
- [RFC2753] R. Yavatkar, D. Pendarakis, R. Guerin, *A Framework for Policy-based Admission Control*, RFC 2753, IETF, January 2000, §2.
- [RFC3198] A. Westerinen et al., *Terminology for Policy-Based Management*, RFC 3198, IETF, November 2001, §3.

- [RFC6410] R. Housley, D. Crocker, E. Burger, *Reducing the Standards Track to Two Maturity Levels*, RFC 6410, IETF, 2011.
- [SCERRI] P. Scerri, D. V. Pynadath, M. Tambe, “Towards Adjustable Autonomy for the Real World”, *JAIR* 17, 171–228, 2002, doi:10.1613/jair.1037.
- [SECPAL] M. Y. Becker, C. Fournet, A. D. Gordon, “SecPAL: Design and semantics of a decentralized authorization language”, *Journal of Computer Security* 18(4), 619–665, 2010, doi:10.3233/JCS-2009-0364.
- [SERGOT] M. Sergot, “A computational theory of normative positions”, *ACM TOCL* 2(4), 581–622, 2001, doi:10.1145/383779.383786.
- [TONTI] G. Tonti, J. M. Bradshaw, R. Jeffers, R. Montanari, N. Suri, A. Uszok, “Semantic Web Languages for Policy Representation and Reasoning: A Comparison of KAoS, Rei, and Ponder”, *ISWC 2003*, LNCS 2870, 419–437, doi:10.1007/978-3-540-39718-2_27.
- [W3C-PROCESS] W3C, *W3C Process Document*, 18 August 2025, §6.3.2 Implementation Experience, https://www.w3.org/policies/process/20250818/, accessed 2026-09-26.
- [X812] ITU-T Recommendation X.812 (11/95), *Information technology — Open Systems Interconnection — Security frameworks for open systems: Access control framework*; identical to ISO/IEC 10181-3:1996. Defines the access control decision function (ADF) and access control enforcement function (AEF). https://www.itu.int/rec/T-REC-X.812-199511-I.
- [XACML] OASIS, *eXtensible Access Control Markup Language (XACML) Version 3.0 Plus Errata 01*, OASIS Standard incorporating Approved Errata, 12 July 2017 (OASIS Standard of 22 January 2013).
- [XACML-CONF] OASIS XACML Technical Committee, *XACML 3.0 conformance tests*, 397 mandatory test cases, each a policy, a request and the expected response; see the XACML TC pages.
- [YAN] Yan, “Do User-Authored Permission Policies Improve Protection Against AI Agent Overreach?”, arXiv:2608.27443, Aug. 2026.